



Pertemuan 6



<https://reactrouter.com/start/declarative/installation>

Kita akan mempelajari React Router, sebuah library di React.js yang digunakan untuk menangani perpindahan halaman (routing) dalam aplikasi single-page.

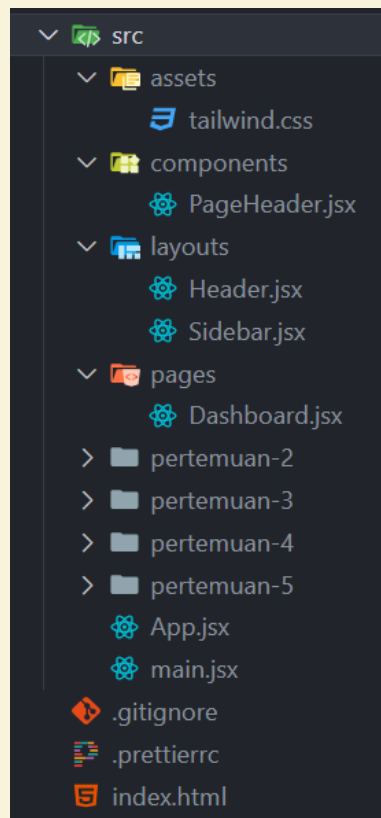


Kita akan menggunakan file `index.html`, `App.jsx` dan `main.jsx` yang ada di direktori terluar, sehingga kita cukup mengakses <http://localhost:5173> tanpa ada tambahan pertemuan.

Persiapkan folder dan file html untuk pertemuan ini dengan cara copy paste dari `pertemuan-5` sebelumnya. Sesuaikan isinya sebagai berikut:

- `index.html` → tetap seperti biasa, berisikan root dan memanggil `main.jsx`
- `App.jsx` → Berisikan kode `<div>` utama yang ada di `pertemuan-5/main.jsx`
- `main.jsx` → Cukup hapus `index.css` yang tidak dibutuhkan

Kedepannya kita akan menggunakan ini tanpa membuat folder pertemuan lagi. Sehingga struktur project untuk pertemuan ini adalah sebagai berikut:



▼ Instalasi dan `<Link>` untuk Basic Route

Setelah project ready, kita akan coba membuat halaman baru `Order` dan `Customer` lalu menerapkan react-route untuk berpindah halaman. Silahkan ikuti langkah-langkah perubahan berikut:

1. Install library `react-router-dom`

```
npm install react-router-dom
```

2. Bungkus `<App/>` di `main.jsx` dengan `<BrowserRouter>`, dan jangan lupa untuk import `BrowserRouter`

```
import { BrowserRouter } from "react-router-dom";
...
...

createRoot(document.getElementById("root"))
  .render(
    <BrowserRouter>
      <App />
    </BrowserRouter>
  )
```

3. Di `App.jsx`, sesuaikan pemanggilan langsung `<Dashboard />` dengan komponen `<Routes>` yang berisi daftar `<Route>`. Dengan cara ini, React Router akan menampilkan komponen yang sesuai berdasarkan nilai `path` dan `element` dari setiap route yang didefinisikan.

Before

```
...
<div className
="flex-1 p-4">
  <Header />

  <Dashboard/>
```

After

```
<div className="flex-1 p-
4">
  <Header />

  <Routes>
    <Route path="/" ele
```

```

    </div>

```

```

    ment={<Dashboard />} />
    <Route path="/order
s" element={<Orders />} />
    <Route path="/custo
mers" element={<Customers /
>} />
  </Routes>
</div>

```

- Di `Sidebar.jsx`, kita akan mengganti tag `<div>` menjadi `<Link>` dan tambahkan properti `to` untuk menentukan path tujuan agar ketika menu diklik, React Router akan mencocokkan path tersebut dengan daftar `<Route>` yang sudah didefinisikan di `main.jsx`, lalu menampilkan komponen yang sesuai:

Before

```

<li>
  <div
    id="menu-1"
    className="hover:text-hijau flex cursor-pointer items-center rounded-xl p-4 font-medium
    text-gray-600 hover:bg-green-200 hover:font-extrabold"
  >
    <MdSpaceDashboard className="mr-4 text-xl" />
    Dashboard
  </div>
</li>

```

After

```

<li>
  <Link
    id="menu-1"
    to="/"
    className="hover:text-hijau flex cursor-pointer items-center rounded-xl p-4 font-medium
    text-gray-600 hover:bg-green-200 hover:font-extrabold"
  >
    <MdSpaceDashboard className="mr-4 text-xl" />
    Dashboard
  </Link>
</li>

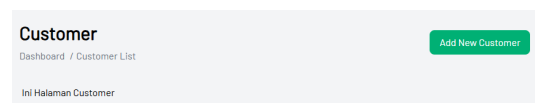
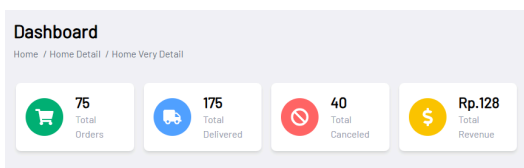
```

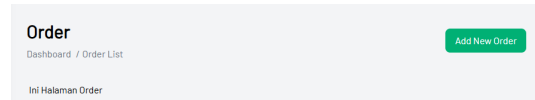
```

import { Link } from "react-router-dom"

```

- Tambah file baru di `pages/` yaitu `Customers.jsx` dan `Orders.jsx` dan isi file tersebut dengan isian sederhana beserta komponen `PageHeader` seperti pada halaman `Dashboard.jsx`
- Silahkan coba untuk berpindah antar halaman `Dashboard`, `Order List` dan `Customer`





▼ Penerapan `<NavLink>` untuk Active State

Saat berpindah halaman, menu Sidebar belum menampilkan state active, ini karena `<Link>` tidak dapat memberikan callback berupa props `isActive` sebagai penanda bahwa komponen yang tampil sesuai dengan path yang sedang diakses.

Kita akan menggunakan `<NavLink>` sehingga active state (`isActive`) bisa kita dapatkan untuk menentukan styling pada menu tersebut;

1. Ubah `<Link>` menjadi `<NavLink>` lalu sesuaikan `className` pada setiap `<NavLink>` menjadi sebuah variabel `menuClass` yang akan mengembalikan class Tailwind yang berbeda tergantung apakah link sedang aktif atau tidak.

```
const menuClass = `flex
cursor-pointer items-center rounded-xl p-4 space-x-2
text-gray-600 hover:text-hijau hover:bg-green-200 hover:font-extrabold`
```

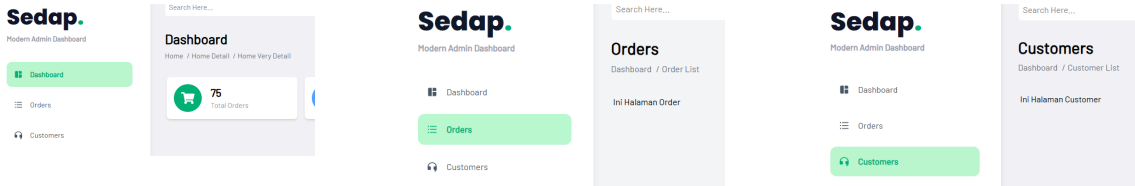
```
<li>
  <NavLink
    id="menu-1"
    to="/"
    className={menuClass}
  >
    <MdSpaceDashboard className="mr-4 text-xl" />
    Dashboard
  </NavLink>
</li>
```

2. Sesuaikan `menuClass` untuk menerima props `isActive` sehingga kita bisa menerapkan kondisi untuk memberikan efek aktif pada menu yang sesuai

```
const menuClass = ({ isActive }) =>
  `flex cursor-pointer items-center rounded-xl p-4 space-x-2
  ${isActive ?
    "text-hijau bg-green-200 font-extrabold" :
  }`
```

```
"text-gray-600 hover:text-hijau hover:bg-green-200 hover:font-extrabold"
}
```

3. Silahkan coba berpindah page, maka menu akan aktif sesuai dengan komponen yang sedang tampil



▼ Route Not Found (404 Page)

Untuk membuat halaman 404 Not Found, kita dapat menerapkan Route sebagai berikut:

```
<Routes>
  <Route path="*" element={<NotFound />} />
  ...
</Routes>
```

Pastikan komponen `<NotFound />` sudah dibuat sebelumnya

Penggunaan `path="*"` menunjukkan bahwa secara default, seluruh url yang diakses selain yang ada pada `<Routes>` maka akan me-render komponen `NotFound`

▼ Latihan

Pisahkan bagian-bagian berikut menjadi komponen tersendiri: dan bisa diakses di setiap halaman

- **List Menu** sudah menjadi Components

- **PageHeader** sudah menjadi Components dan menerima props berikut:
 - `title`
 - `breadcrumb` (string atau array):
 - `children`
- Pilih 1 Desain Figma untuk menampilkan halaman Error 404, lalu buat Komponen `ErrorPage` dengan desain tersebut dan terapkan props berikut agar komponen tersebut bisa diterapkan untuk error lain seperti 400, 401, dan 403:
 - Kode Error
 - Deskripsi Error
 - Gambar Error

Lalu siapkan 3 ListMenu di Sidebar untuk menampilkan Error 400, Error 401 dan Error 403

- Silahkan isi halaman Customers dan Orders dengan data JSON yang berbeda yang terdiri dari 30 data dengan atribut berikut:

Orders:

- Order ID
- Customer Name
- Status (Pending / Completed / Cancelled)
- Total Price
- Order Date

Customer:

- Customer ID
- Customer Name
- Email
- Phone
- Loyalty (Bronze/Silver/Gold)

Tombol "Add Orders" dan "Add Customer" yang ada di PageHeader akan menampilkan Form sesuai data JSON